STRUCTURES

ALVINA ANJUM



LECTURE OUTLINE

- Abstract Data type
- Structures
- Accessing Elements of Structures
- Comparing Structures Variables

ABSTRACT DATA TYPES

- Abstract data types (ADTs) are data types created by the programmer.
- ADTs have their own range (or domain) of data and their own sets of operations that may be performed on them.
- An abstract data type (ADT) is a data type created by the programmer and is composed of one or more primitive data types.

STRUCTURE

- C++ allows you to group several variables together into a single item known as a structure.
- So far you've written programs that keep data in individual variables.
  If you need to group items together,
- C++ allows you to create arrays. The limitation of arrays, however, is that all the elements must be of the same data type.
- Sometimes a relationship exists between items of different types. For example, a payroll system might keep the variables shown in

STRUCTURES

- These variables hold data for a single employee.

Variable Definition	Data Held
`int empNumber;`	Employee number
`string name;`	Employee's name
`double hours;`	Hours worked
`double payRate;`	Hourly pay rate
`double grossPay;`	Gross pay

CREATING STRUCTURE

- Before a structure can be used, it must be declared. Here is the general format of a structure declaration:

```
struct tag
{
   variable declaration;
   // ... more declarations
   //     may follow...
};
```

- The *tag* is the name of the structure
- It is used as a data type
- The variable declarations that appear inside the braces declare *members* of the structure

EXAMPLE

- Here is an example of a structure declaration that holds the payroll data

```
struct PayRoll
{
    int empNumber;     // Employee number
    string name;       // Employee's name
    double hours;      // Hours worked
    double payRate;    // Hourly pay rate
    double grossPay;   // Gross pay
};
```

- It’s important to note that the structure declaration in our example does not define a variable.

DEFINING STRUCTURE

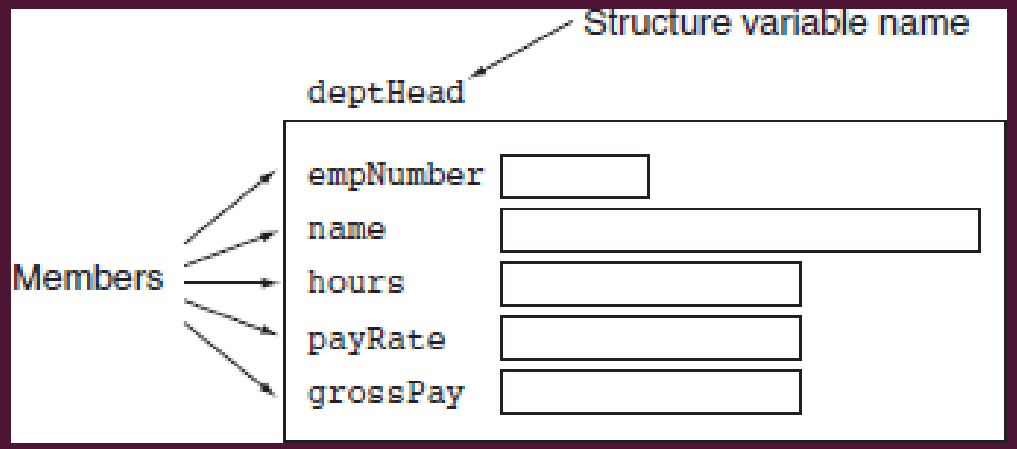
- It simply tells the compiler what a PayRoll structure is made of.
- In essence, it creates a new data type named PayRoll .
- You can define variables of this type with simple definition statements, just as you would with any other data type. For example, the following statement defines a variable named deptHead :

```
PayRoll deptHead;
```

- The data type of deptHead is the PayRoll structure. The structure tag, PayRoll , is listed before the variable name just as the word int or double would be listed to define variables of those types.

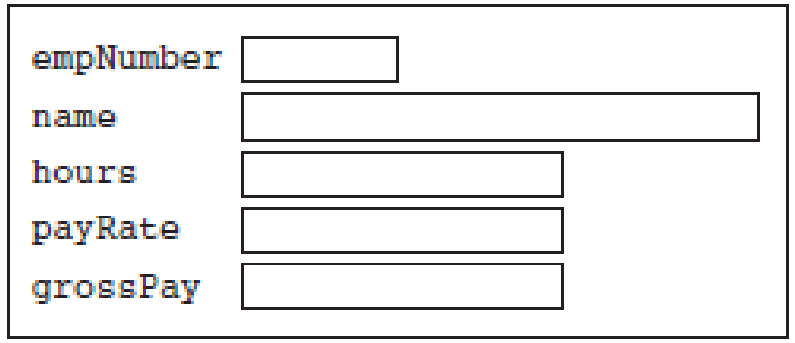
Payroll Structure

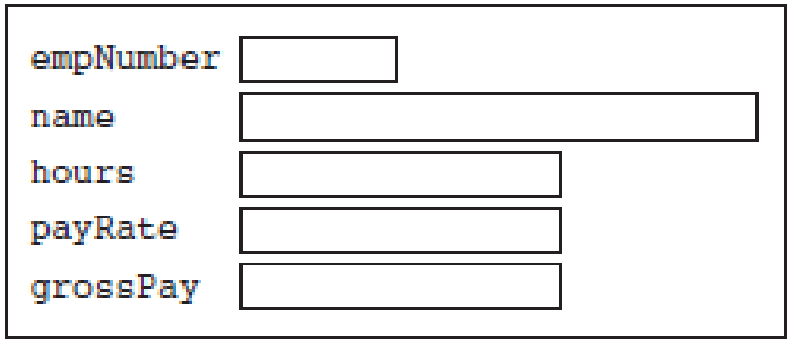
DEFINING STRUCTURE

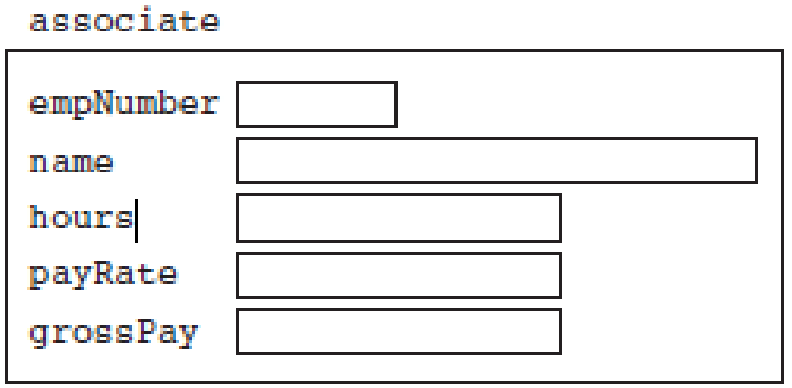


- Just as it’s possible to define multiple int or double variables, it’s possible to define multiple structure variables in a program. The following statement defines three PayRoll variables:
- deptHead , foreman , and associate :

```
PayRoll deptHead, foreman, associate;
```







ACCESSING THE ELEMENTS OF STRUCTURE

- **The *dot operator* (.) allows you to access structure members in a program.**
- C++ provides the dot operator (a period) to access the individual members of a structure.
- Using our example of deptHead as a PayRoll structure variable, the following statement demonstrates how to access the empNumber member:

```
deptHead.name = "Alvina";
```

- In this statement, the number 475 is assigned to the empNumber member of deptHead .
- The dot operator connects the name of the member variable with the name of the structure variable it belongs to.
- The following statements assign values to the empNumber members of the deptHead , foreman , and associate structure variables:

```
deptHead.hours = 30;
foreman.empNumber = 897;
associate.empNumber = 729;
cout<<deptHead.hours;
```

- With the dot operator you can use member variables just like regular variables. For example these statements display the contents of deptHead ’s members:

```
cout << foreman.empNumber << endl;
cout << deptHead.name << endl;
cout << deptHead.hours << endl;
cout << deptHead.payRate << endl;
cout << deptHead.grossPay << endl;
```

COMPARING STRUCTURE VARIABLES

- You cannot perform comparison operations directly on structure variables. For example, assume that circle1 and circle2 are Circle structure variables.
- The following statement will cause an error.

```
if (circle1 == circle2) // Error!
```

- In order to compare two structures, you must compare the individual members, as shown in the following code.

```
if (circle1.radius == circle2.radius &&
circle1.diameter == circle2.diameter &&
circle1.area == circle2.area)
```

INITIALIZING STRUCTURE VARIABLE

- The members of a structure variable may be initialized with starting values when the structure variable is defined.
- A structure variable may be initialized when it is defined, in a fashion similar to the initialization of an array. Assume the following structure declaration exists in a program:

```
struct CityInfo
{
string cityName;
string state;
long population;
int distance;
};
```

- A variable may then be defined with an initialization list, as

- You do not have to provide initializers for all the members of a structure variable. For example, the following statement only initializes the cityName member of location :

```
CityInfo location = {"Tampa"};
```

- The state , population , and distance members are left uninitialized. The following statement only initializes the cityName and state members, while leaving population and distance uninitialized:

```
CityInfo location = {"Atlanta", "GA"};
```

- If you leave a structure member uninitialized, you must leave all the members that follow it uninitialized as well. C++ does not provide a way to skip members in a structure.
- For example, the following statement, which attempts to skip the initialization of the population member, is *not* legal:

```
CityInfo location = {"Knoxville", "TN",60000,56}; // Illegal!
```

NEXT LECTURE

- Arrays of Structures
- Nested Structures

STRUCTURES AND ARRAYS

ALVINA ANJUM



OUTLINE

- Arrays as Members of Structures
- Array of Structures
- Nested Structures

ARRAY AS A MEMBER OF A STRUCTURE

```
struct std {
string name;
int reg_no;
int marks[5];
}s1;
    getline(cin,s1.name);
    cin>>s1.reg_no;
    for(int i =0;i<5;i++)
    cin>>s1.marks[i];
```

```
Main(){ std s1;
}
```

ARRAY OF STRUCTURE

- **CONCEPT: Arrays of structures can simplify some programming tasks.**
- An array of structures is defined like any other array. Assume the following structure declaration exists in a program:

```
struct BookInfo
{
string title;
string author;
string publisher;
double price;
};
```

WHAT IS AN ARRAY OF STRUCTURES?

n **array of structures** is a collection of multiple structure variables stored in an array.

Instead of one book:

BookInfo b1;

You can store multiple books:

BookInfo books[5];

Declaration

BookInfo books[5];

Creates an array of 5 books

Each element has:

- title
- author
- publisher
- price

ACCESSING ELEMENTS

ACCESSING INDIVISUAL BOOK

```
books[0].title = "C++ Programming";
books[0].author = "Bjarne Stroustrup";
books[0].publisher = "Pearson";
books[0].price = 500;
```

GENERIC SYNTAX

array[index].member

USING LOOPS

```
for(int i = 0; i < 5; i++) {
    cin >> books[i].title;
    cin >> books[i].author;
    cin >> books[i].publisher;
    cin >> books[i].price;
}
```

DISPLAYING DATA

```
for(int i = 0; i < 5; i++) {
    cout << books[i].title << endl;
    cout << books[i].author << endl;
    cout << books[i].publisher << endl;
    cout << books[i].price << endl;
}
```

DEFINING NESTED STRUCTURE

- When a structure contains another structure, it is called nested structure. For example, we have two structures named Address and Employee. To make Address nested to Employee, we have to define Address structure before and outside Employee structure and create an object of Address structure inside Employee structure.

```
struct structure1
{
      - - - - - - - - - - -
      - - - - - - - - - - -
};

struct structure2
{
      - - - - - - - - - - -
      - - - - - - - - - - -
      struct structure1 obj;
```

EXAMPLE

```
struct Address
{
      char HouseNo[25];
      char City[25];
      char PinCode[25];
};

struct Employee
{
   int Id;
   char Name[25];
   float Salary;
   struct Address Add;
};
```

```
int i;
Employee E;

cout << "\n\tEnter Employee Id : ";
cin >> E.Id;

cout << "\n\tEnter Employee Name : ";
cin >> E.Name;

cout << "\n\tEnter Employee Salary : ";
cin >> E.Salary;

cout << "\n\tEnter Employee House No : ";
cin >> E.Add.HouseNo;

cout << "\n\tEnter Employee City : ";
cin >> E.Add.City;

cout << "\n\tEnter Employee House No : ";
cin >> E.Add.PinCode;
```

STRUCTURES AND FUNCTIONS

ALVINA ANJUM



LECTURE OUTLINE

- Structures as Function Arguments
- Reference Parameter
- Returning Structures from Functions
- Pointers to Structures

STRUCTURE AS FUNCTION ARGUMENTS

CONCEPT: Structure variables may be passed as arguments to functions.

- Like other variables, the individual members of a structure variable may be used as function arguments. For example, assume the following structure declaration exists in a program:

```
struct Rectangle
{
double length;
double width;
double area;
};
```

- Let's say the following function definition exists in the same program:

```
double multiply(double x, double y)
{
return x * y;
}
```

STRUCTURE AS FUNCTION ARGUMENTS

Assuming that the box is an object variable of rectangle structure type, the following function call will pass box.length in x and box.width into y

- The return value will be stored in box.area .

```
box.area = multiply(box.length, box.width);
```

Sometimes it's more convenient to pass an entire structure variable into a function instead of individual members. For example, the following function definition uses a Rectangle structure variable as its parameter:

```
void showRect(Rectangle r)
{
cout << r.length << endl;
cout << r.width << endl;
cout << r.area << endl;
```

- The following function call passes the box variable into r :

```
showRect(box);
```

STRUCTURE AS REFERENCE PARAMETERS

- Structures can be passed by reference just as other simple types.
- When a structure is passed by reference the called function declares a reference for the passed structure and refers to the original structure elements through its reference.
- Thus, the called function works with the original values.

PASSING STRUCTURE TO FUNCTION

```
void displayData(Person);
int main()
{
 Person p;
.....
..... displayData(p);
return 0;
}
void displayData(Person p)
{.....
    }
```

RETURNING STRUCTURE FROM FUNCTION

```
Person getData(Person p);  // Function declaration

int main(){
   Person p;
   p = getData(p);   // Calling function and assigning returned object to
p
   ...
   return 0;
}

Person getData(Person p){
   ...
   return p;  // Returns modified or initialized object
}
```

NOTE

- The value of all members of a structure variable can be assigned to another structure using assignment operator **=** if both structure variables are of same type. You don't need to manually assign each members.

POINTERS AND STRUCTURES

- A pointer variable can be created not only for native types like (int, float, double etc.) but they can also be created for user defined types like structure.

```
#include
<iostream> using
namespace std;

struct temp
    { int i;
    float f;
};

int main()
    { temp
    *ptr;
    return 0;
}
```

POINTERS AND STRUCTURES

```
#include <iostream>
using namespace std;

struct Distance {
   int feet;
   float inch;
};

int main() {
   Distance *ptr, d;
   ptr = &d;

   cout << "Enter feet: ";    cin >> (*ptr).feet;

   cout << "Enter inch: ";  cin >> (*ptr).inch;

   cout << "\nDisplaying information:" << endl;
   cout << "Distance = " << (*ptr).feet << " feet " << (*ptr).inch << " inches" << endl;

   return 0;
}
```

EXPLANATION

- In this program, a pointer variable ptr and normal variable d of type structure Distance is defined.
- The address of variable d is stored to pointer variable, that is, ptr is pointing to variable d. Then, the member function of variable d is accessed using pointer.

NOTE

- Since pointer ptr is pointing to variable d in this program, (*ptr).inch and d.inch is exact same cell. Similarly, (*ptr).feet and d.feet is exact same cell.
- The syntax to access member function using pointer is ugly and there is alternative notation **->** which is more common.
- ptr->feet is same as (*ptr).
- feet ptr->inch is same as (*ptr).inch

```
cin >> ptr->feet;
cin >> ptr->inch;
cout << "Distance = " << ptr->feet << " feet " << ptr->inch << " inches";
```